Package 'metronautica'

October 24, 2026

Type Package

Title Wearable Triaxial Accelerometer Metrology and Signal Processing

Version 0.1.0

Date 2026-10-22

Language en-GB

NeedsCompilation no

Author Millen James Theophilus [aut, cre]
(<https://orcid.org/0000-0001-9823-7177>)

Maintainer Millen James Theophilus <m.j.theophilus@lboro.ac.uk>

Description Native wearable triaxial accelerometer parsing, signal diagnostics, clipping, static and dynamic characterisation, filtering and reporting. Metrological terminology follows the International Vocabulary of Metrology, JCGM (2012) <doi:10.59161/JCGM200-2012>. Standalone activity counts implement the published ActiGraph algorithm of Neishabouri et al. (2022) <doi:10.1038/s41598-022-16003-x>, with its published filter as the default and configurable band-pass cutoffs for modified counts. Developed for the wearable-accelerometer metrology research article under submission for publication, with accompanying analyses and data in WAM <https://github.com/miltheo/wam>.

License GPL-3

URL https://github.com/miltheo/metronautica

BugReports https://github.com/miltheo/metronautica/issues

Encoding UTF-8

Depends R (>= 4.1.0)

Imports
tibble, dplyr (>= 1.0.0), data.table, digest, jsonlite, signal, ggplot2 (>= 3.4.0), ggrepel (>= 0.9.1), scales, stats, utils, grid, tools

Suggests plotly, GENEAread, GGIRread, read.gt3x, MIMSunit, TSEntropies

Collate 'm_qc.R'
'm_parsers.R'
'm_accel_clipper.R'
'm_static.R'
'm_dynamic.R'
'm_filter.R'

'm_accel_summary_suite.R'
'm_agcounts.R'
'm_plotter.R'
'm_sample_data.R'

Contents

metronautica-package . . . 3
accel_meta . . . 4
accel_metric_agcounts . . . 4
accel_metric_ai . . . 6
accel_metric_enmo . . . 8
accel_metric_mad . . . 9
accel_metric_mims . . . 10
accel_metric_odba . . . 11
accel_metric_rocam . . . 13
accel_metric_svmbp . . . 14
accel_metric_svmg . . . 15
accel_metric_vm . . . 16
accel_metric_z_angle . . . 16
accel_metric_z_angle_change . . . 17
allan_nonoverlap . . . 18
as_accel_tbl . . . 19
attach_qc . . . 20
clip_bundle . . . 20
clip_dynamic_stages . . . 21
common-data-format . . . 22
compute_accel_summaries . . . 23
compute_ai_noise_var_from_static . . . 28
dm_apply_vm_calibration_bundle . . . 29
dm_apply_vm_calibration_tbl . . . 30
dm_fit_vm_calibration . . . 30
dynamic_metrology_report . . . 31
filter_accel_tbl . . . 35
invalidate_qc . . . 36
measurement-definitions . . . 36
m_dynamic_response . . . 38
m_load_sample_data . . . 39
parse_raw_auto . . . 40
plot_accel_clip . . . 42
plot_processed_metric . . . 42
plot_static_allan . . . 43
plot_static_drift_vm . . . 43
qc_accel_tbl . . . 44
static_1hz_means . . . 46
static_allan . . . 46
static_allan_minima_wide . . . 47
static_axis_noise_metrics . . . 48
static_clip . . . 49
static_drift . . . 49
static_metrology_from_tbl . . . 50
static_noise_summary_from_tbl . . . 51

static_offset_vm_5min 53

Index **54**

`metronautica-package` *Metronautica: Navigate wearable triaxial accelerometer signals into transparent metrology and reports.*

Description

Native wearable triaxial accelerometer parsing, signal diagnostics, static and dynamic characterisation, feature extraction and reporting.

Details

Use the linked guides for workflows and this reference for function behaviour. Filtering and calibration are explicit choices. The package returns recording-level outcomes; WAM supplies cohort inference and inter-device equivalence analyses.

Repositories

Developed for the wearable-accelerometer metrology research article under submission. Package source: https://github.com/miltheo/metronautica. Associated analyses, observations and native-recording catalogue: https://github.com/miltheo/wam.

Guides

Getting Started provides recording, characterisation, feature and calibration workflows; Function map connects stages to source files; Feature comparison shows the shared-trial example. The offline example below needs no external recordings or optional packages; parsing attaches primary-five diagnostics.

See Also

`metronautica-tibble` (the sample-level data contract), `measurement-definitions`

Examples

```
library(metronautica)
# Load the installed recording.
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
# Defaults: ENMO and MAD.
# Select features explicitly.
features <- compute_accel_summaries(x, measures = c("ENMO", "MAD"),
    epoch_s = 5, quiet = TRUE)
# Inspect values and units.
head(features$data_wide)
features$settings$metric_units
```

accel_meta *Read acceleration metadata*

Description

Read attached metadata without modifying acceleration.

Usage

```
accel_meta(x)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.

Value

The meta attribute, normally a list; NULL if absent.

See Also

as_accel_tbl, common-data-format

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
accel_meta(x)$acquisition
```

accel_metric_agcounts *Compute ActiGraph-compatible or modified activity counts*

Description

Standalone published-filter activity counts, with optional custom band-pass cutoffs and explicit gap/boundary handling.

Usage

```
accel_metric_agcounts(tbl, epoch_s = 1, fs_hz,
    lower_hz = NULL, upper_hz = NULL, filter_order = 4,
    lfe = FALSE, gap_policy = c("error", "zero", "carry_forward"),
    vm_round = TRUE, start_time = NULL, end_time = NULL,
    boundary = c("error", "pad"), pad_value = NULL,
    filter_guard_s = 60)
```

Arguments

`tbl`	Acceleration table or data/full bundle with POSIXct time and finite numeric ax_g, ay_g, az_g (g). Requires strict timestamp order on the nominal sampling grid; observed zero samples are retained.
`epoch_s`	Positive whole-second epoch duration. Accumulate axes before VM calculation. Discard incomplete trailing raw epochs before rate conversion; record discarded sample count.
`fs_hz`	Explicit nominal recording rate (Hz): 30, 40, 50, 60, 70, 80, 90, 100, 32, 64, 128 or 256. Gap-spanning timestamp estimates cannot substitute for acquisition settings.
`lower_hz, upper_hz`	Both NULL retain exact published coefficients, rather than substituting nominal cutoffs. Supplying both selects modified causal Butterworth filtering with -3-dB edges satisfying 0 < lower_hz < upper_hz < min(15, fs_hz/2). Internal processing remains 30 Hz.
`filter_order`	Custom Butterworth prototype order, integer 1–8 (default 4); ignored for the published filter. See Calculation stages in the counts guide for transfer order and gain matching.
`lfe`	Enable the published low-frequency-extension threshold rule (default FALSE), without changing filter coefficients. Quantisation rules are in the counts guide.
`gap_policy`	error (default) rejects missing interior sample positions; zero inserts zero acceleration; carry_forward repeats the last observed triaxial sample. Fill values are computational conventions, not measured missing acceleration. NA/Inf axes error rather than becoming gaps.
`vm_round`	TRUE rounds sqrt(Axis1 squared + Axis2 squared + Axis3 squared) to the nearest integer, matching the R comparator; FALSE retains unrounded VM. Summing one-second VM differs from accumulating axes before VM calculation.
`start_time, end_time`	Single POSIXct bounds, inclusive start/exclusive end; defaults: first sample and last sample plus one nominal interval. Epochs begin at resolved start_time without civil-clock rounding. Metadata distinguishes native start from first observation. Process continuous input before selecting epochs when filter history matters.
`boundary`	error (default) rejects bounds beyond coverage; pad permits explicitly requested end padding with required pad_value. Native-header bounds are never applied automatically; padding does not establish vendor compatibility.
`pad_value`	Three finite X/Y/Z accelerations (g) for boundary = "pad", used at both requested ends. Padding counts/locations are reported; no implicit constant-first-sample startup padding.
`filter_guard_s`	Non-negative flagging interval after imputation/initialisation (default 60 s). Centred-FIR power-of-two conversion also flags the second before imputation and the final processing second. Flags do not change counts. This context guard is not a validated settling time or uncertainty bound; lower custom cutoffs may need longer guards.

Details

Defaults use the published coefficient-defined filter. Custom cutoffs replace only that stage and produce modified counts, without inherited behavioural-threshold validation. Calculation stages

and validation documents rate conversion, quantisation, precision, boundary effects and agreement results.

Value

Tibble: time, Axis1 (input Y), Axis2 (input X), Axis3 (input Z), Vector.Magnitude and value (same VM). Per-epoch input_samples, missing_samples and padded_samples count nominal positions; imputed marks inserted values; affected_by_imputation includes the filter guard; boundary_affected marks initialisation/finite-record context. Flags concern this call, not prior imputation. Explicit fill policies retain counts rather than NA; inspect flags before interpretation.

The settings attribute records reference, exact coefficients, filter, available precision provenance, interval, gaps, padding, discarded samples, rounding and quality conventions.

Validation scope

Default axis counts matched the pinned Python reference exactly for 3,763,659 paired one-second values. ActiLife export agreement depends on precision, startup context and gaps; see the validation guide.

References

Neishabouri et al. (2022). Quantification of acceleration as activity counts in ActiGraph wearable. Scientific Reports 12, 11958. doi:10.1038/s4159802216003x.

Published coefficients: ActiGraph/agcounts, GPL-3.0, commit 3137c9b9a26539ae2d9645d396586fe15faece59. https://github.com/actigraph/agcounts. Numerical coefficients are acknowledged from that source; the R implementation is provided by Metronautica.

See Also

parse_raw_auto, qc_accel_tbl, compute_accel_summaries

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
counts <- accel_metric_agcounts(x, epoch_s = 5, fs_hz = 100)
head(counts)
```

---

accel_metric_ai *Compute Activity Index*

Description

Noise-adjusted one-second Activity Index, aggregated by epoch.

Usage

```
accel_metric_ai(tbl, epoch_s, fs_hz, anchor = "center", ai_noise_var
    = NULL, ai_noise_mode = c("supplied", "window_start",
    "quiet_segments"),
    ai_noise_window_s = 30, ai_quiet_quantile = 0.1, ai_scale =
    c("absolute", "relative"), ai_epoch_summary = c("sum", "mean"))
```

Arguments

tbl	Acceleration table or parser/clip bundle; see common-data-format.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
fs_hz	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
anchor	Output timestamp selected from the one-second bin starts: first for start, last for end, or the middle bin start (ceiling of half the bin count) for center. This differs from sample-level anchors of ENMO/MAD and does not change the calculated values.
ai_noise_var	Mean-axis stationary noise variance (g squared), required by default for AI; estimate with compute_ai_noise_var_from_static. Zero disables noise subtraction.
ai_noise_mode	supplied (default) requires ai_noise_var. With NULL variance, window_start or quiet_segments enables estimation; supplied variance takes precedence. The initial window must be stationary; quiet selection does not prove stationarity. Failed estimation errors, without substituting zero.
ai_noise_window_s	Initial noise-window duration (s), for window_start only.
ai_quiet_quantile	quiet_segments selection quantile, strictly between zero and one.
ai_scale	Absolute Activity Index in g (default) or dimensionless noise-relative Activity Index.
ai_epoch_summary	Epoch sum (default) or mean of one-second Activity Index values.

Details

Calculate sample variance per axis within one-second windows, average the three variances, subtract the stationary mean-axis noise variance, truncate negative excess to zero and take the square root. The absolute index is in g. Relative scaling divides excess variance by noise variance before the square root; zero noise variance gives undefined relative values. Aggregate the one-second indices by the selected sum or mean.

Value

A tibble with epoch_id, time and value. Absolute output is in g; relative output is dimensionless. Sum/mean aggregate one-second indices, not acceleration samples. The settings attribute records noise variance/source, estimation settings, scale, aggregation, rate and duration; ai_noise_var_used also records the variance in g squared.

References

Bai et al. (2016), doi:10.1371/journal.pone.0160644.

See Also

compute_accel_summaries

`accel_metric_enmo` *Compute Euclidean norm minus one*

Description

Epoch mean of max(VM - 1 g, 0).

Usage

```
accel_metric_enmo(tbl, epoch_s, anchor = "center", out_unit =
    c("mg", "g"))
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration (s); see the pipeline epoch conventions in `compute_accel_summaries`.
`anchor`	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
`out_unit`	Output unit: g, or mg (result multiplied by 1000).

Details

Negative differences are truncated to zero before epoch averaging.

Value

Tibble: `time` (selected epoch label), `epoch_id` (elapsed-time bin, starting at one), `value` (selected g or mg unit).

See Also

`compute_accel_summaries`

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
out <- accel_metric_enmo(x, epoch_s = 5)
head(out)
```

`accel_metric_mad` *Compute mean amplitude deviation*

Description

Epoch mean absolute deviation of VM from its epoch mean.

Usage

```
accel_metric_mad(tbl, epoch_s, anchor = "center", out_unit = c("mg",
    "g"))
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration (s); see the pipeline epoch conventions in `compute_accel_summaries`.
`anchor`	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
`out_unit`	Output unit: g, or mg (result multiplied by 1000).

Details

For epoch VM values v, calculate mean(abs(v - mean(v))).

Value

Tibble: `time` (selected epoch label), `epoch_id` (elapsed-time bin, starting at one), `value` (selected g or mg unit).

References

Vaha-Ypya et al. (2015), A universal, accurate intensity-based classification of different physical activities using raw data of accelerometer. doi:10.1111/cpf.12127.

See Also

`compute_accel_summaries`

`accel_metric_mims` *Compute MIMS-unit*

Description

MIMS-unit output using configurable upstream MIMSunit processing.

Usage

```
accel_metric_mims(tbl, epoch_s, dynamic_range_g, mims_filter_type =
    "butter", mims_cutoffs = c(0.2, 5), mims_use_extrapolation =
    TRUE, mims_use_filtering = TRUE, mims_combination = "sum",
    mims_allow_truncation = TRUE)
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration in seconds, passed as the upstream MIMS epoch setting. MIMSunit controls epoch boundaries, timestamps and treatment of partial epochs; this helper does not impose the common elapsed-sample epoch index.
`dynamic_range_g`	Configured peak-to-peak axis span (g): 16 means -8 to +8 g. Required by the direct MIMS helper; the summary function accepts NULL only with valid acquisition metadata. Missing range errors; observed values never define it.
`mims_filter_type`	MIMSunit::custom_mims_unit filter: butter (default), ellip or bessel, as supported upstream; used with mims_cutoffs.
`mims_cutoffs`	Filter cutoffs in Hz; default c(0.2, 5). Use c(0.2, 10) explicitly for the altered passband.
`mims_use_extrapolation`	Enable MIMS saturation/extrapolation (TRUE). FALSE skips that step but retains upstream grid interpolation. This concerns acceleration, not behavioural-time extrapolation.
`mims_use_filtering`	Enable internal MIMS band-pass filtering; FALSE bypasses it and defines a different pipeline.
`mims_combination`	MIMS axis combination: sum (default) adds axis summaries; vm takes their vector magnitude.
`mims_allow_truncation`	Allow upstream MIMS to set small summaries to zero; unrelated to native-recording clipping.

Details

Call MIMSunit::custom_mims_unit with triaxial g inputs and configured axis limits. Default 0.2-5 Hz and altered 0.2-10 Hz passbands define different pipelines.

Value

A tibble with `time` (upstream MIMS epoch labels), `epoch_id` (sequential row number), and `value` (the combined MIMS-unit output per epoch). These values are not g or mg. The `settings` attribute records measurement span/limits, filter and cutoffs, extrapolation, truncation, combination, duration and MIMSunit version.

References

John et al. (2019), An open-source monitor-independent movement summary for accelerometer data processing. doi:10.1123/jmpb.20180068.

See Also

`compute_accel_summaries`

`accel_metric_odba` *Compute dynamic body acceleration*

Description

Epoch means of overall, vectorial or partial dynamic body acceleration.

Usage

```
accel_metric_odba(tbl, epoch_s, fs_hz, anchor = "center", output =
    c("ODBA", "VEDBA", "PDBA"), static_dynamic_method =
    c("centered_running_mean", "low_pass_static_filter",
    "high_pass_dynamic_filter"), stroke_window_rule = c("fixed_3s",
    "custom_window_s", "stroke_period_1x", "stroke_period_2x",
    "sensitivity_windows"), running_mean_window_s = 3,
    stroke_period_s = NA_real_, lowpass_cutoff_hz = 0.2,
    highpass_cutoff_hz = 0.2, filter_type = "butterworth",
    filter_order = 4, pdba_axes = c("x", "y"), out_unit = c("mg",
    "g"), edge_handling = "partial_centered_window",
    missing_data_policy = "mean_na_rm")
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration (s); see the pipeline epoch conventions in `compute_accel_summaries`.
`fs_hz`	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
`anchor`	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
`output`	ODBA sums absolute dynamic components across all three axes; VEDBA uses their Euclidean norm; PDBA sums the selected absolute components. Each is averaged within the output epoch.

`static_dynamic_method`
: Axis separation method. centered_running_mean subtracts the centred local axis mean; low_pass_static_filter subtracts a low-pass axis estimate; high_pass_dynamic_filter uses high-pass-filtered axes directly.

`stroke_window_rule`
: Window selection for running-mean separation. fixed_3s uses three seconds; custom_window_s uses running_mean_window_s; stroke_period_1x/2x use max(3, stroke_period_s) or max(3, 2*stroke_period_s). sensitivity_windows uses the requested main window and permits a separate sensitivity summary.

`running_mean_window_s`
: Requested centred averaging-window duration in seconds. Used by custom_window_s and the main result of sensitivity_windows; fixed_3s overrides it. Rounded to samples using fs_hz.

`stroke_period_s`
: Positive representative stroke/movement-cycle period in seconds, required for stroke_period_1x/2x. The function does not estimate a movement period.

`lowpass_cutoff_hz`
: Low-pass cutoff in Hz when static_dynamic_method is low_pass_static_filter. Filter each axis and subtract that static estimate before forming the feature.

`highpass_cutoff_hz`
: High-pass cutoff in Hz when static_dynamic_method is high_pass_dynamic_filter. The filtered axes are the dynamic components.

`filter_type`
: Implementation identifier: butterworth is supported for low-/high-pass separation. This is not an automatic choice among filter families.

`filter_order`
: Order passed to signal::butter for the forward/reverse separation filter. The double-pass frequency response differs from a single pass.

`pdba_axes`
: PDBA axes: x, y and/or z; default x/y sums their absolute dynamic components. ODBA/VEDBA are unaffected.

`out_unit`
: Output unit: g, or mg (result multiplied by 1000).

`edge_handling`
: Supported rule partial_centered_window: near signal boundaries, average only the available samples within the centred running window; no external padding is added.

`missing_data_policy`
: Supported rule mean_na_rm: ignore missing observations when calculating local means and epoch means; all-missing estimates remain undefined. This is not gap repair.

Details

Separate static/dynamic axes by the selected implemented method, form ODBA, VEDBA or PDBA, then average within epochs. The centred-window method uses partial available windows at boundaries. A different separation window/filter defines a different pipeline.

Value

Tibble: `time` (selected epoch label), `epoch_id` (elapsed-time bin, starting at one), `value` (selected g or mg unit).

Output settings

The `odba_settings` attribute records the selected metric, separation method, requested/resolved window duration, cycle-period rule, filter settings, selected axes, output unit and missing-data/edge policies. The direct helper returns one selected metric, not a list of multiple features. The summary function provides separately requested ODBA, VEDBA and PDBA columns.

See Also

`compute_accel_summaries`

`accel_metric_rocam` *Compute rate-of-change acceleration movement*

Description

Epoch mean Euclidean sample-to-sample triaxial acceleration change, after running-median smoothing.

Usage

```
accel_metric_rocam(tbl, epoch_s, fs_hz, anchor = "center", median_s
    = 1)
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration (s); see the pipeline epoch conventions in `compute_accel_summaries`.
`fs_hz`	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
`anchor`	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
`median_s`	Running-median duration (s), converted with fs_hz to an odd sample count of at least three; smooths the difference series before aggregation.

Details

Default smoothing is a one-second running median of the difference series. No time division or one-second resampling: units are g/sample change, not g/s.

Value

Tibble: `time` (selected sample label), `epoch_id` (elapsed-time bin, starting at one), `value` (g per successive-sample change, without division by elapsed time).

See Also

`compute_accel_summaries`

`accel_metric_svmbp` *Compute band-pass vector magnitude*

Description

Epoch summaries of VM recombined after axis band-pass filtering.

Usage

```
accel_metric_svmbp(tbl, epoch_s, fs_hz, anchor = "center",
    cutoffs_hz = c(0.2, 15), order = 4, output = c("mean_mg",
    "mean_g", "sum_mg", "sum_g", "g_seconds"), nyquist_action =
    c("stop", "clip"), quiet = FALSE)
```

Arguments

`tbl`	Acceleration table or parser/clip bundle; see `common-data-format`.
`epoch_s`	Positive epoch duration (s); see the pipeline epoch conventions in `compute_accel_summaries`.
`fs_hz`	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
`anchor`	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
`cutoffs_hz`	Lower and upper axis band-pass cutoffs in Hz, applied before vector-magnitude recombination. The upper cutoff must lie below fs_hz/2 unless clipping is explicitly selected.
`order`	Axis band-pass Butterworth order; default four, applied forward/reverse before VM recombination.
`output`	Band-pass VM aggregation: mean_mg and mean_g return epoch means; sum_mg and sum_g return sample sums; g_seconds divides the g sample sum by fs_hz. Sample sums depend on the sample count.
`nyquist_action`	stop (default) rejects upper cutoffs at/above Nyquist; clip adjusts them to 99 percent of Nyquist.
`quiet`	Suppress informational messages, retaining errors.

Details

Filter each acceleration axis using signal::butter and forward/reverse filtfilt, recombine the filtered axes as VM, then aggregate. Axis filtering precedes recombination; this differs from filtering VM or using the compensated standalone filter.

Value

A tibble with `time`, `epoch_id` and `value`. Units are mg/g for means, mg/g sample sums for sum_mg/sum_g, or g seconds for g_seconds. The `settings` attribute records requested/applied cutoffs, filter order, sampling frequency, Nyquist policy, output aggregation, epoch duration and anchor.

See Also

compute_accel_summaries

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
out <- accel_metric_svmbp(x, epoch_s = 5, fs_hz = 100,
    cutoffs_hz = c(.69, 10), quiet = TRUE)
head(out)
```

---

accel_metric_svmg *Compute gravity-subtracted vector magnitude*

Description

Epoch integral, sample sum or mean of VM minus 1 g, optionally rectified.

Usage

```
accel_metric_svmg(tbl, epoch_s, fs_hz, anchor = "center", output =
    c("g_seconds", "sample_sum_g", "mean_mg", "mean_g"), rectify =
    TRUE)
```

Arguments

tbl	Acceleration table or parser/clip bundle; see common-data-format.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
fs_hz	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
anchor	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
output	g_seconds integrates the gravity-subtracted series by sum/fs_hz; sample_sum_g returns its sample sum; mean_mg and mean_g return its epoch mean in mg/g.
rectify	TRUE takes abs(VM - 1); FALSE retains the signed VM-minus-1-g series. This differs from the non-negative truncation used by ENMO.

Details

SVMGS, SVMGSEC and SVMGMEAN use absolute VM-minus-1-g differences. Their sample sum, frequency-normalised integral and epoch mean are distinct outputs.

Value

Tibble: time (selected sample label), epoch_id (elapsed-time bin, starting at one), value (g seconds for g_seconds, g sample sums for sample_sum_g, mg for mean_mg, or g for mean_g).

See Also

compute_accel_summaries

accel_metric_vm *Compute vector magnitude*

Description

Epoch mean, sample SD, maximum or 95th percentile of VM.

Usage

```
accel_metric_vm(tbl, epoch_s, anchor = "center", summary_fun =
    c("mean", "sd", "max", "p95"), out_unit = c("g", "mg"))
```

Arguments

tbl	Acceleration table or parser/clip bundle; see common-data-format.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
anchor	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
summary_fun	Within-epoch statistic of sample VM: mean, sample SD, maximum, or the R type-7 95th percentile.
out_unit	g retains the VM statistic; mg multiplies it by 1000.

Details

Compute sample VM as sqrt(ax_g^2 + ay_g^2 + az_g^2), then the selected epoch statistic.

Value

Tibble: time (selected epoch label), epoch_id (elapsed-time bin, starting at one), value (selected g or mg unit).

See Also

compute_accel_summaries

accel_metric_z_angle *Compute Z-axis angle*

Description

Epoch mean or sample SD of the full-vector Z-angle.

Usage

```
accel_metric_z_angle(tbl, epoch_s, anchor = "center", output =
    c("mean_deg", "sd_deg"))
```

Arguments

tbl	Acceleration table or parser/clip bundle; see common-data-format.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
anchor	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
output	mean_deg returns the epoch mean Z-angle; sd_deg returns its sample SD. Both are in degrees.

Details

For each sample compute atan(z/sqrt(x^2+y^2)) in degrees, then take the selected epoch mean or SD. No low-pass gravity-vector estimate is introduced; this is the full-vector orientation feature.

Value

Tibble: time (selected sample label), epoch_id (elapsed-time bin, starting at one), value (degrees).

See Also

compute_accel_summaries

---

accel_metric_z_angle_change

Compute change in Z-axis angle

Description

Epoch mean or maximum absolute successive change in smoothed Z-angle.

Usage

```
accel_metric_z_angle_change(tbl, epoch_s, fs_hz, anchor = "center",
    median_s = 5, output = c("mean_abs_change_deg",
    "max_abs_change_deg"))
```

Arguments

tbl	Acceleration table or parser/clip bundle; see common-data-format.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
fs_hz	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
anchor	Observed-sample label: first (start), last (end), or ceiling of half the sample count (center). Labels do not change values.
median_s	Running-median duration (s), converted with fs_hz to an odd sample count of at least three; smooths the change/orientation series before aggregation.
output	mean_abs_change_deg averages successive absolute differences of the smoothed Z-angle within each epoch; max_abs_change_deg returns their maximum. Neither is divided by elapsed time.

Details

Compute full-vector Z-angle, apply the selected running median, then form successive absolute differences. Summarise the differences within epochs. It measures change per successive sample, not angular velocity.

Value

Tibble: `time` (selected sample label), `epoch_id` (elapsed-time bin, starting at one), `value` (degrees per successive-sample change, without division by elapsed time).

See Also

`compute_accel_summaries`

`allan_nonoverlap` *Compute non-overlapping Allan deviation of a time series*

Description

Allan deviation from successive non-overlapping cluster means.

Usage

```
allan_nonoverlap(x, fs_hz, tau_s, min_M = 4L)
```

Arguments

`x`	Numeric one-axis sequence (g) at the supplied sampling frequency; not a table/bundle.
`fs_hz`	Finite positive sampling frequency (Hz), used for sample windows or time integrals.
`tau_s`	Requested averaging times (s), rounded to sample counts. Duplicate counts and unsupported long windows are omitted.
`min_M`	Minimum cluster count per averaging time (default four). Shorter supported times yield more adjacent-cluster differences.

Value

A tibble containing tau in seconds and adev in the input signal unit.

Method

Round tau_s to sample counts; form complete non-overlapping clusters and calculate sqrt(mean(diff(cluster_means)^2)/2). Omit unsupported lengths and trailing incomplete clusters. Returned tau equals cluster sample count/fs_hz.

See Also

`static_allan`, `measurement-definitions`

`as_accel_tbl` *Construct a Metronautica Tibble*

Description

Construct the four-column sample-level table defined by Metronautica Tibble 0.1.0, retaining optional metadata.

Usage

```
as_accel_tbl(time, ax_g, ay_g, az_g, meta = list())
```

Arguments

`time`	POSIXct sample timestamps. UTC is the Metronautica Tibble storage convention; existing instants and timezone attributes are retained.
`ax_g`	Numeric X-axis acceleration in g.
`ay_g`	Numeric Y-axis acceleration in g.
`az_g`	Numeric Z-axis acceleration in g.
`meta`	Optional pipeline metadata list, empty by default. Its fields are not required by the core data contract; see `metronautica-tibble`.

Details

Axes must have the same length as time; scalar recycling is rejected. Non-finite observations and timestamp defects are retained for diagnostics. The constructor does not sort, resample, calibrate or infer acceleration units. The constructor preserves supplied metadata without estimating acquisition settings. The configured axis full-scale range is the peak-to-peak span: 16 g denotes +/-8 g.

Value

An `accel_tbl` inheriting from `tbl_df`, with exactly `time`, `ax_g`, `ay_g`, `az_g` and a `meta` attribute.

See Also

`metronautica-package`

Examples

```
t0 <- as.POSIXct("2026-01-01 00:00:00", tz = "UTC")
fs <- 100
s <- (0:5999) / fs
meta <- list(acquisition = list(fs_derived_hz = fs,
    fs_nominal_hz = fs, full_scale_range_g = 16))
x <- as_accel_tbl(t0 + s, 0.1 * sin(2 * pi * 2 * s),
    0.1 * cos(2 * pi * 2 * s), 1 + 0.1 * sin(2 * pi * s), meta)
class(x)
qc_accel_tbl(x)$n_samples
```

attach_qc *Attach signal diagnostics to recording metadata*

Description

Compute and attach current primary-five signal diagnostics.

Usage

```
attach_qc(x, scope = "whole parsed recording")
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.
scope	Text describing the recording interval to which the diagnostics apply.

Value

The same input container, with primary-five diagnostics in meta$processing$qc. Bundle and data metadata are synchronised.

See Also

qc_accel_tbl, invalidate_qc

clip_bundle *Select a time interval with provenance*

Description

Select an inclusive time interval from an acceleration table or parser bundle.

Usage

```
clip_bundle(obj, start_time = NULL, duration_s = NULL, end_time =
    NULL, trim_pad_s = 0, keep_qc = FALSE)
```

Arguments

obj	An acceleration table, parser bundle with data, or dynamic bundle with full; see Stage bundles for the scope of clipping.
start_time	Inclusive UTC bounds (POSIXct or parseable text). Supply a start with an end or duration, or an end with a duration.
duration_s	Duration in seconds; used to infer a missing bound. Explicit start and end take precedence.
end_time	Inclusive UTC bounds (POSIXct or parseable text). Supply a start with an end or duration, or an end with a duration.
trim_pad_s	Seconds added before the start and after the end; a negative value contracts the interval.
keep_qc	Retain inherited diagnostics if TRUE. They are not recomputed and may describe the original recording.

Details

Both endpoints are included. A 10-s interval sampled at 100 Hz can therefore contain 1001 samples. This differs deliberately from the exclusive end used by static and dynamic stage clipping. The input is unchanged. Invalid or empty intervals raise errors. Recompute `qc_accel_tbl()` for the selected interval.

Value

The same object form as the input. Acquisition bounds are updated and a clipping event is appended to `meta$processing$history`. Inherited QC and a top-level bundle QC entry are removed by default.

Stage bundles

Only `full` is clipped; stored `stages`/`schedule` remain unchanged. Reconstruct stages before metrology. Unlike clipping, `dm_apply_vm_calibration_bundle` transforms every stored signal.

See Also

`static_clip`, `clip_dynamic_stages`, `attach_qc`

`clip_dynamic_stages` *Select trimmed excitation stages*

Description

Build a timed excitation schedule and select trimmed, end-exclusive stages.

Usage

```
clip_dynamic_stages(x, start_utc, n_stages, stage_min,
    buffer_min, rpm_start, rpm_step, trim_start_min, trim_end_min,
    keep_empty = FALSE, verbose = FALSE,
    time_offset_s = 0)
```

Arguments

`x`	Acceleration table or parser/clip bundle; see `common-data-format`.
`start_utc`	UTC window start; POSIXct or parseable text.
`n_stages`	Required positive integer stage count; not inferred.
`stage_min`	Required raw stage duration (min), before trimming.
`buffer_min`	Required non-negative inter-stage interval (min); excluded from stages, retained in full.
`rpm_start`	Required first-stage RPM label: supplied excitation, not measured frequency.
`rpm_step`	Required RPM-label increment; assumes a regular schedule.
`trim_start_min`	Required non-negative start trim (min). Combined start/end trims must be less than stage_min.
`trim_end_min`	Required non-negative end trim (min); changes the selected interval, not the native file.

keep_empty	Retain empty tables if TRUE; otherwise store NULL at the corresponding stage position.
verbose	Print processing messages.
time_offset_s	Additive schedule-anchor shift in seconds; sample timestamps are unchanged.

Details

All schedule parameters are required. Input is timestamp-sorted; inspect whole-recording QC first. Stages use $[start, end)$. First raw start = start_utc + time_offset_s; subsequent starts differ by stage_min + buffer_min. Empty stages retain their positions; resolve them before pairing.

Value

A list with `full` (untrimmed full-assessment table), `stages` (named list, e.g. `rpm_50`), `schedule` (raw and trimmed UTC bounds, settings and labels), and `meta`. Each clip has freshly computed QC.

Processing order

Filter before constructing stages. `filter_accel_tbl` changes only an existing bundle's full recording; response uses stages and some diagnostics use full, so both must represent the intended signal state.

See Also

dynamic_metrology_report, filter_accel_tbl, clip_bundle

---

common-data-format *Metronautica Tibble 0.1.0*

Description

The sample-level acceleration data contract used by Metronautica.

Details

Only four data columns are required, in this order:

time Sample timestamps of class POSIXct. UTC is the storage convention; the constructor preserves supplied instants and timezone.

ax_g, ay_g, az_g Numeric acceleration on the device X, Y and Z axes in g; 1 g = 9.80665 m/s squared.

Columns have equal length, with one row per decoded observation. Missing/non-finite observations and timestamp defects are retained for diagnostics. Structural conformance does not establish signal quality or suitability for an analysis.

`as_accel_tbl()` returns an `accel_tbl` inheriting from `tbl_df`. No metadata field, class tag, version field, diagnostic or attachment is required for core conformance. The full versioned specification and optional field catalogue are in data-contract/0.1.0/SPECIFICATION.md in the repository.

Optional pipeline implementations

Optional `meta` contains `file`, `acquisition` and `processing` sections, read by `accel_meta()`. Parser bundles use `list(data = x, meta = accel_meta(x))`; table metadata takes precedence over bundle metadata. Individual analyses may require settings or explicit arguments beyond core conformance.

Parsing attaches primary-five diagnostics by default; `as_accel_tbl()` does not. Use `attach_qc()` for decoded samples. Diagnostics are the recommended minimum workflow assessment; unknown range makes near-limit results unavailable.

Static/noise and dynamic report functions default to `attach_report = TRUE`. Capture `result$data` to retain assessments at `meta$processing$reports$static` or `dynamic`; FALSE adds no record. Records retain results, assessed source/selection, settings, diagnostics and provenance; plots and raw signal copies remain separate. Records become historical after package clipping/filtering/calibration. External edits can leave stale records. Assessment schema `"1"` is versioned separately from this contract.

RDS retains typed samples and attachments. Keep native files, unfiltered samples, provenance and feature settings for later metric calculations when their input requirements are met. Epoch summaries cannot reconstruct original samples.

Input and return contracts

Raw-signal helpers accept four-column acceleration tables or data/full bundles. Unwrapping does not select, sort or repair samples. Table metadata takes precedence over top-level meta.

Container operations static_clip returns a table; clip_dynamic_stages creates a stage bundle. Generic clipping, QC, filtering and calibration preserve their documented input form.

Stage bundles filter_accel_tbl and clip_bundle change only full; existing stages remain unchanged. Perform these operations before constructing stages. Calibration application processes full and stages.

Derived inputs static_drift and static_offset_vm_5min require one-second means; Allan-minimum/plot helpers require an Allan table; allan_nonoverlap requires a numeric axis series. Dynamic reporting/fitting requires corresponding named stage bundles; m_dynamic_response also accepts numeric VM vectors. Parsers/loaders take paths or sample selectors.

Calculation helpers return derived results rather than the original container.

See Also

`as_accel_tbl`, `qc_accel_tbl`, `compute_accel_summaries`, `getting-started`

`compute_accel_summaries` *Compute epoch-level acceleration features*

Description

Compute time-domain summaries through explicitly configured processing pipelines.

Usage

```
compute_accel_summaries(accel_input, measures = c("ENMO", "MAD"),
    epoch_s = 5, epoch_anchor = c("center", "start", "end"), fs_hz =
    NULL, target_fs = NULL, dynamic_range_g = NULL, vm_summary =
    c("mean", "sd", "max", "p95"), vm_unit = c("g", "mg"), enmo_mg =
    TRUE, mad_mg = TRUE, odba_static_dynamic_method =
    c("centered_running_mean", "low_pass_static_filter",
    "high_pass_dynamic_filter"), odba_stroke_window_rule =
    c("fixed_3s", "custom_window_s", "stroke_period_1x",
    "stroke_period_2x", "sensitivity_windows"),
    odba_running_mean_window_s = 3, odba_stroke_period_s = NA_real_,
    odba_lowpass_cutoff_hz = 0.2, odba_highpass_cutoff_hz = 0.2,
    odba_filter_type = "butterworth", odba_filter_order = 4,
    pdba_axes = c("x", "y"), odba_unit = c("mg", "g"),
    odba_sensitivity_windows_s = c(1, 2, 3, 5, 10),
    odba_edge_handling = "partial_centered_window",
    odba_missing_data_policy = "mean_na_rm", svmg_output =
    c("g_seconds", "sample_sum_g", "mean_mg", "mean_g"),
    svmg_rectify = TRUE, svmbp_cutoffs = c(0.2, 15), svmbp_order =
    4, svmbp_output = c("mean_mg", "mean_g", "sum_mg", "sum_g",
    "g_seconds"), svmbp_nyquist_action = c("stop", "clip"),
    rocam_median_s = 1, ai_noise_var = NULL, ai_noise_mode =
    c("supplied", "window_start", "quiet_segments"), ai_noise_window_s = 30,
    ai_quiet_quantile = 0.1, ai_scale = c("absolute", "relative"),
    ai_epoch_summary = c("sum", "mean"), z_angle_output =
    c("mean_deg", "sd_deg"), z_angle_change_median_s = 5,
    z_angle_change_output = c("mean_abs_change_deg",
    "max_abs_change_deg"), mims_filter_type = "butter", mims_cutoffs
    = c(0.2, 5), mims_use_extrapolation = TRUE, mims_use_filtering
    = TRUE, mims_combination = "sum", mims_allow_truncation = TRUE,
    ag_lfe = FALSE, ag_preprocess = c("none", "regularise",
    "paper30", "antialias100"), ag_anchor = c("floor_sec",
    "round_sec"), ag_antialias_cutoff_hz = NULL, ag_filter_order =
    4, quiet = FALSE)
```

Arguments

accel_input	Acceleration table or parser/clip bundle with data/full; see common-data-format.
measures	Case-insensitive feature identifiers. Defaults: ENMO and MAD. Other choices are listed under Feature identifiers.
epoch_s	Positive epoch duration (s); see the pipeline epoch conventions in compute_accel_summaries.
epoch_anchor	Label each epoch by its first (start), last (end) or middle (center) observed sample. Upstream implementations may retain different anchors; inspect timestamps before joining.
fs_hz	Explicit input rate (Hz); otherwise acquisition metadata, then median positive timestamp interval. target_fs controls separate interpolation. Neither setting establishes regularity.
target_fs	Optional positive target rate in Hz. Linear interpolation is applied when it differs from the input rate; this alone is not an anti-aliasing filter.

`dynamic_range_g`
: Configured peak-to-peak axis span (g); NULL uses acquisition metadata. Required for MIMS; see `accel_metric_mims`.

`vm_summary`
: VM epoch statistic: mean, sample SD, maximum or 95th percentile.

`vm_unit`
: VM output unit: g or mg.

`enmo_mg`
: ENMO output in mg (TRUE) or g (FALSE).

`mad_mg`
: MAD output in mg (TRUE) or g (FALSE).

`odba_static_dynamic_method`
: Axis static/dynamic separation method; see `accel_metric_odba` for all body-acceleration settings below.

`odba_stroke_window_rule`
: Running-mean window rule: fixed, custom, stroke-period-based or sensitivity_windows.

`odba_running_mean_window_s`
: Custom/main sensitivity running-mean duration (s); fixed_3s overrides it.

`odba_stroke_period_s`
: Supplied cycle period (s) for stroke-period rules; minimum averaging window 3 s.

`odba_lowpass_cutoff_hz`
: Static-component low-pass cutoff (Hz), for low_pass_static_filter.

`odba_highpass_cutoff_hz`
: Dynamic-component high-pass cutoff (Hz), for high_pass_dynamic_filter.

`odba_filter_type`
: Separation filter: butterworth.

`odba_filter_order`
: Separation Butterworth order; unused by running-mean methods.

`pdba_axes`
: PDBA axes: x, y and/or z (default x/y).

`odba_unit`
: Body-acceleration epoch means in mg or g.

`odba_sensitivity_windows_s`
: Alternative running-mean durations (s); results in settings$odba$sensitivity_summary.

`odba_edge_handling`
: partial_centered_window: average available boundary samples.

`odba_missing_data_policy`
: mean_na_rm: omit missing observations from means.

`svmg_output`
: Inactive compatibility argument; SVMGS/SVMGSEC/SVMGMEAN fix aggregation.

`svmg_rectify`
: Rectify VM-minus-1-g; see `accel_metric_svmg`.

`svmbp_cutoffs`
: Custom axis band-pass edges (Hz); named variants fix cutoffs. See `accel_metric_svmbp`.

`svmbp_order`
: Axis band-pass Butterworth order (forward/reverse).

`svmbp_output`
: Band-pass aggregation: epoch mean, sample sum or time integral; units follow the selected identifier.

`svmbp_nyquist_action`
: stop rejects invalid upper cutoffs; clip limits them to 99 percent of Nyquist.

`rocam_median_s`
: Change-series running-median duration (s); see `accel_metric_rocam` for sample rounding and units.

`ai_noise_var` Stationary mean-axis noise variance (g squared); zero disables subtraction. See `accel_metric_ai`.

`ai_noise_mode` supplied requires variance; window_start/quiet_segments enables estimation when variance is NULL. Supplied variance takes precedence.

`ai_noise_window_s`
Initial noise-window duration (s), for window_start.

`ai_quiet_quantile`
Quiet-selection quantile, strictly between zero and one.

`ai_scale` absolute: g; relative: dimensionless noise-relative index.

`ai_epoch_summary`
Epoch sum or mean of one-second AI values.

`z_angle_output` Epoch mean or sample SD in degrees; see `accel_metric_z_angle`.

`z_angle_change_median_s`
Z-angle running-median duration (s); see `accel_metric_z_angle_change`.

`z_angle_change_output`
Epoch mean or maximum absolute successive Z-angle change (degrees).

`mims_filter_type`
MIMS filter family; see `accel_metric_mims` for all MIMS settings below.

`mims_cutoffs` MIMS band-pass cutoffs (Hz), default c(0.2, 5).

`mims_use_extrapolation`
Enable upstream saturation/extrapolation processing.

`mims_use_filtering`
Enable internal MIMS band-pass filtering.

`mims_combination`
Combine axis summaries by sum or vector magnitude (vm).

`mims_allow_truncation`
Allow upstream small-value truncation to zero.

`ag_lfe`, `ag_preprocess`, `ag_anchor`, `ag_antialias_cutoff_hz`, `ag_filter_order`
Inactive compatibility arguments. Use `accel_metric_agcounts` for counts.

`quiet` Suppress informational messages, retaining errors.

Details

Input follows `metronautica-tibble` and is timestamp-sorted; epoch conventions below describe the feature pipeline. Feature-specific methods, limits and return settings are defined by the direct functions below. MIMS requires optional MIMSunit and a configured measurement span; AI requires noise variance unless estimation is selected. Inspect QC before processing; gaps are not repaired.

Value

Invisible list: `data_wide` (time, one column per measure), `data_long` (time, measure, value), `settings` (resolved rate, input provenance, range, processing).

Settings include metric_units; bandpass (requested/applied cutoffs, order, rate, Nyquist handling); dynamic_range_g, dynamic_range_limits_g and dynamic_range_source (argument/metadata/unavailable). Requested mims settings record configuration/dependency version; ai records noise variance/source, applicable estimation options, scale and aggregation. Unrequested feature settings are NULL.

No behavioural classification is returned. Save the call alongside settings: not every argument is recorded.

Feature identifiers

Identifier	Direct function	Default output
VM	accel_metric_vm	Mean, g
ENMO	accel_metric_enmo	Mean, mg
MAD	accel_metric_mad	Mean, mg
ODBA	accel_metric_odba	Mean, mg
VEDBA	accel_metric_odba	Mean, mg
PDBA	accel_metric_odba	Mean, mg
SVMGS	accel_metric_svmg	Sample sum, g
SVMGSEC	accel_metric_svmg	Integral, g seconds
SVMGMEAN	accel_metric_svmg	Mean, mg
SVMBP	accel_metric_svmbp	Mean, mg
SVMBP_02_15	accel_metric_svmbp	Mean, mg
SVMBP_069_10	accel_metric_svmbp	Mean, mg
ROCAM	accel_metric_rocam	Mean change, g
AI	accel_metric_ai	Sum of 1-s AI, g
ZANGLE	accel_metric_z_angle	Mean, degrees
ZANGLE_CHANGE	accel_metric_z_angle_change	Mean change, degrees
MIMS	accel_metric_mims	MIMS-units per epoch

The SVM gravity-subtracted identifiers fix their aggregation; `svmg_rectify = FALSE` instead retains signed differences. SVMBP uses `svmbp_cutoffs`; the named variants fix their cutoffs, while `svmbp_output` still selects their aggregation. The direct helpers define processing alternatives. AG is not an available selector.

Pipeline rate and epoch conventions

These are optional pipeline conventions, not required fields of the Metronautica Tibble contract. Feature rate precedence is explicit `fs_hz`, metadata, then timestamps. Neither nominal nor timestamp-derived rate independently validates a clock. Filters and index pairing also require suitable regularity or synchronisation.

Unless documented otherwise, positive `epoch_s` defines elapsed-time bins from the first sample, retaining the final partial epoch. Different durations define different pipelines. `anchor` selects an observed sample: start = first, end = last, center = ceiling of half the epoch sample count. Labels do not affect values. Activity Index uses one-second bin starts; MIMS retains upstream conventions; activity counts have separate boundaries. The dispatcher sorts timestamps; `as_accel_tbl()` does not.

Epoch timestamps and joins

Features can use different epoch labels. data_long preserves each feature's timestamps; data_wide joins exact timestamps, potentially creating extra rows and unpaired NA cells. These need not indicate missing acceleration; do not zero-fill them. See the worked epoch-label example.

See Also

The feature-settings example demonstrates explicit configuration. See Feature comparison for the dynamic comparison.

Examples

```
# Load the installed recording.
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
# Defaults: ENMO and MAD.
# Select features explicitly.
features <- compute_accel_summaries(x,
    measures = c("VM", "ENMO", "MAD", "SVMBP_069_10"),
    epoch_s = 5, fs_hz = 100, quiet = TRUE)
# Inspect values and units.
head(features$data_wide)
features$settings$metric_units
```

---

compute_ai_noise_var_from_static

Estimate stationary variance for the Activity Index

Description

Mean-axis sample variance across complete stationary windows.

Usage

```
compute_ai_noise_var_from_static(x, window_s = 1, fs_hz = NULL,
    use_middle_prop = 1)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.
window_s	Non-overlapping window duration (s); round(fs_hz * window_s) must be at least two samples. Average axis sample variances within windows, then across complete windows.
fs_hz	NULL derives Hz from the median finite positive timestamp interval; otherwise use the supplied finite positive Hz to convert window_s to samples.
use_middle_prop	Central fraction retained before windowing, in (0, 1]: 1 uses all input; 0.8 uses the middle 80 percent. Stationarity is not tested.

Value

A list with ai_noise_var (mean per-axis stationary variance, g squared), ai_noise_sd (its square root, g), window_values (per-window variances), n_windows, fs_hz and window_s. Trailing incomplete sample windows are discarded. No finite estimates or fewer than two samples per window raises an error.

Method

Average (var(x)+var(y)+var(z))/3 across complete windows for AI noise correction. Unlike the maximum-axis noise floor, this uses sample variances without detrending. No calibration or stationarity detection is performed.

See Also

accel_metric_ai, static_axis_noise_metrics

Examples

```
seconds <- (0:5999) / 100
x <- as_accel_tbl(as.POSIXct("2026-01-01", tz = "UTC") + seconds,
    .001 * sin(2*pi*7*seconds), .001 * cos(2*pi*7*seconds),
    1 + .001 * sin(2*pi*11*seconds))
noise <- compute_ai_noise_var_from_static(list(data = x), window_s = 1)
noise[c("ai_noise_var", "ai_noise_sd", "n_windows")]
```

---

dm_apply_vm_calibration_bundle

Apply vector-magnitude calibration to a stage bundle

Description

Apply a supplied VM mapping to every stored signal in a bundle.

Usage

```
dm_apply_vm_calibration_bundle(clips, cal_row)
```

Arguments

clips	Acceleration table, parser bundle with data, or dynamic bundle with full and stages. Every present signal is calibrated; NULL stages remain NULL.
cal_row	One-row table or named scalar list: finite vm_calibration_intercept, vm_calibration_slope, vm_calibration_quadratic (zero for linear). Optional vm_calibration_direction must be ref_on_test. Invalid/ambiguous rows error.

Value

The same table or bundle form. Apply identical coefficients to full/data and each non-NULL stage. Per-table metadata and top-level meta record the operation; inherited QC is removed from all modified signals and the top-level bundle.

Method

Map test VM v to max(b0+b1*v+b2*v^2, 0); scale each axis by corrected_VM/v, using one for zero/undefined VM. This preserves vector direction, rather than calibrating axes independently. Source decoding provenance is retained separately from added calibration history.

See Also

dm_fit_vm_calibration, attach_qc, common-data-format

dm_apply_vm_calibration_tbl *Apply vector-magnitude calibration to triaxial samples*

Description

Apply a supplied scalar VM mapping while preserving vector direction.

Usage

```
dm_apply_vm_calibration_tbl(df, cal_row)
```

Arguments

df	Acceleration table or parser/clip bundle with POSIXct time and numeric axes in g. Container type and timestamps are preserved.
cal_row	One-row table or named scalar list: finite vm_calibration_intercept, vm_calibration_slope, vm_calibration_quadratic (zero for linear). Optional vm_calibration_direction must be ref_on_test. Invalid/ambiguous rows error.

Value

The same input container type, with calibrated axes and unchanged timestamps. A parser bundle retains its data/meta structure; a stage bundle also calibrates its stages. Metadata records the applied row and processing event, marks the signal calibrated and removes pre-calibration QC. Call attach_qc to obtain diagnostics for the changed signal.

Method

Map test VM v to max(b0+b1*v+b2*v^2, 0); scale each axis by corrected_VM/v, using one for zero/undefined VM. This preserves vector direction, rather than calibrating axes independently. Source decoding provenance is retained separately from added calibration history.

See Also

dm_fit_vm_calibration, attach_qc, common-data-format

dm_fit_vm_calibration *Fit a reference-based vector-magnitude calibration*

Description

Select a linear or quadratic mapping between paired test/reference VM values.

Usage

```
dm_fit_vm_calibration(test_clips, ref_clips, window_s = 5, select_by
    = c("AIC", "AICc", "BIC", "linear"), fit_unit = c("window",
    "sample"), direction = c("ref_on_test", "test_on_ref"))
```

Arguments

test_clips	Test-device stage bundle returned by clip_dynamic_stages.
ref_clips	Reference-device stage bundle with matching stage names.
window_s	Paired-VM averaging-bin duration (s) for fit_unit = window; unused for sample fitting.
select_by	Select the linear or quadratic candidate using AIC, corrected AIC (AICc), or BIC; "linear" forces the linear candidate.
fit_unit	Fit paired window-mean or sample VM. Pairing follows stage/sample-index rules, without timestamp alignment.
direction	ref_on_test maps test to reference VM for application. test_on_ref reverses the regression for inspection only; do not apply it as a correction.

Value

A list with summary (one-row coefficients/model-selection results), fit_linear and fit_quadratic (lm objects), chosen_fit and pairs (test_vm/reference_vm fitting data). Insufficient pairs yield no fitted model and NA coefficients; application rejects those coefficients.

Method

Pair matched-stage VM using evenly spaced sample indices. Fit linear/quadratic candidates in the requested direction and select by criterion, or force linear. Within-recording fitting requires independent validation; apply only ref_on_test coefficients.

See Also

Calibration example, dm_apply_vm_calibration_tbl, dm_apply_vm_calibration_bundle

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
reference <- x
reference[c("ax_g", "ay_g", "az_g")] <-
    1.1 * reference[c("ax_g", "ay_g", "az_g")]
stage <- function(x) list(full = x, stages = list(rpm_50 = x))
fit <- dm_fit_vm_calibration(stage(x), stage(reference),
    select_by = "linear", fit_unit = "sample")
fit$summary
```

---

dynamic_metrology_report

Characterise response relative to a reference

Description

Estimate VM response slope, signed VM bias, integral non-linearity and pooled composite error against a reference.

Usage

```
dynamic_metrology_report(test_clips, ref_clips, test_axis_fsr_g =
    NULL, ref_axis_fsr_g = NULL, window_s = 5, epoch_time = c("end",
    "center", "start"), compute_entropy = TRUE, entropy_m = 2,
    entropy_r = NULL, entropy_tau = 1, verbose = FALSE, progress =
    TRUE, add_plot = TRUE, inspect_alignment = TRUE, align_hz = 10,
    align_max_lag_s = 7200, align_by = c("vm", "ax", "ay", "az"),
    align_plot_stages_n = 3, compute_stage_xcorr = FALSE,
    comp_plot_max_n = 20000L, comp_plot_ylim = NULL,
    attach_report = TRUE)
```

Arguments

test_clips	Test-device stage bundle returned by clip_dynamic_stages.
ref_clips	Reference-device stage bundle with matching stage names.
test_axis_fsr_g	Test-axis peak-to-peak span (g); NULL uses acquisition metadata. Supplied values must be finite positive scalars; invalid values error without fallback. Required for range-normalised errors. Example: 16 for +/-8 g.
ref_axis_fsr_g	Reference-axis peak-to-peak span (g); NULL uses acquisition metadata. Supplied values must be finite positive scalars. Missing range remains unavailable; test-error normalisation does not require it. Example: 16 for +/-8 g.
window_s	Within-stage VM averaging-bin duration (s), for repeatability_error_cv; separate from pooled sample-level fitting and trial-to-trial repeatability.
epoch_time	Retained end/center/start metadata setting; does not change within-stage repeatability bins.
compute_entropy	Compute diagnostic sample/approximate entropy through optional TSEntropies. Disabled/unavailable implementations return NA; not an equivalence decision.
entropy_m	Embedding dimension passed as dim to SampEn/ApEn; default two, on standardised VM.
entropy_r	Matching tolerance in standardised-VM units; NULL uses 0.2 times sample SD. Not an acceleration equivalence margin.
entropy_tau	Embedding lag passed to TSEntropies; default one sample, with duration dependent on sampling rate.
verbose	Print processing messages.
progress	Show a text progress bar.
add_plot	Return diagnostic ggplot objects.
inspect_alignment	Estimate diagnostic lags without correcting timestamps or response metrics.
align_hz	Uniform diagnostic resampling rate (Hz) for lag estimation; response/error inputs are unchanged.
align_max_lag_s	Maximum cross-correlation lag magnitude (s), default 7200. Repetitive excitation can produce ambiguous peaks; estimated lags are not applied.
align_by	Lag-inspection channel: vm (recombined axes), ax, ay or az.

`align_plot_stages_n`
: Inactive compatibility argument; does not select stages or limit plots/calculations.

`compute_stage_xcorr`
: Also estimate matched-stage lags when inspect_alignment is TRUE, separately from whole-trial diagnostics. No lag is applied.

`comp_plot_max_n`
: Maximum displayed composite-trace samples, selected at evenly spaced indices. Fitting/errors use all data.

`comp_plot_ylim`
: Composite-trace display limits: lower/upper g pair, or NULL for observed range. Fitting is unaffected.

`attach_report`
: TRUE (default) appends a compact assessment to processing$reports$dynamic on returned data and the test_clips tables. FALSE adds no record. Input objects and reference tables are not modified in place.

Details

OLS regresses test VM on reference VM. Unequal lengths use nearest positions on an evenly spaced sample-index grid, without timestamp alignment or continuous interpolation. Stage and pooled matching differ: the pooled reference is length-matched once after concatenation. Select synchronous, suitable stages and assess timing sensitivity; meta records pairing policy/range sources.

With F the test-axis peak-to-peak span, stage VM integral non-linearity uses $\sqrt{3}F$; pooled composite error uses $\sqrt{3}F/2$. Neither is the nonnegative VM output range. Absolute VM bias is abs(mean(test - reference)); percentage scale-factor error is 100(slope - 1).

Frequency diagnostics are included; frequency-domain features and inter-device equivalence tests are in WAM (https://github.com/miltheo/wam).

Value

A list: `stages` (one row per paired stage), `overall` (pooled response/composite error), `plots` (ggplot objects, possibly empty), and `meta` (ranges, windows, stage names and alignment diagnostics). Slope and scale-factor error are dimensionless; intercept/bias/SD are in g. VM integral non-linearity (retained field `linearity_error_vs_reference`) and composite error are percentages. Within-stage epoch-mean VM CV (retained field `repeatability_error_cv`) and other CV/utilisation fields are fractions. Trial-to-trial repeatability requires repeated assessments and separate aggregation.

Returned stage fields

The returned `data` is the full test clip when available, otherwise its data table or the concatenated matched test stages. `test_clips` retains the input bundle structure with optional assessments attached to its tables and synchronised metadata. Attached results include stage/overall tables and test/reference diagnostics with reference provenance; shared record fields and retention follow common-data-format.

Here F is the test-axis peak-to-peak span; v and r are paired test/reference VM in g.

stage, rpm Matched stage name and its parsed numerical RPM label.

fs_hz_test, fs_hz_ref, n Timestamp-derived sampling frequencies in Hz and the number of test samples.

vm_mean, vm_standard_deviation, vm_mean_amplitude_deviation Mean, sample SD and mean absolute deviation from the stage mean of v, all in g.

vm_coefficient_of_variation Sample SD divided by mean VM, a fraction; undefined for zero mean.

signal_power_vm Mean of squared sample VM, in g squared, including its mean component. This is a signal mean square, not physical mechanical power.

dominant_frequency Maximum-magnitude Fourier bin of demeaned stage VM (Hz). The grid has floor(N/2) entries spanning zero to half the derived rate; N is the finite series length. This whole-stage diagnostic differs from five-second pooled-triaxial features.

dominant_frequency_magnitude Maximum absolute Fourier coefficient of demeaned stage VM over the retained half spectrum. No sample-count, single-sided-amplitude or spectral-density normalisation is applied. It scales with record length; it is neither calibrated sinusoidal amplitude nor power spectral density.

sample_entropy, approximate_entropy Dimensionless SampEn/ApEn of standardised VM with the supplied parameters; NA when disabled, unavailable or unsuccessful.

vm_bias_vs_reference Mean signed v minus r, in g; not mean absolute paired error.

response_slope, response_intercept, scale_factor_error OLS test-on-reference slope (dimensionless), intercept (g) and slope minus one (fraction).

linearity_error_vs_reference 100 times the maximum absolute fitted residual divided by sqrt(3) F, a percentage.

snr_time_domain Mean VM divided by sample SD of VM. A linear dimensionless ratio, not decibels or a separately measured signal/noise decomposition.

snr_frequency_domain Squared dominant Fourier magnitude divided by the sum of squared remaining retained magnitudes. A linear dimensionless ratio; zero remainder gives NA.

repeatability_error_cv SD divided by mean of within-stage window-mean VM, a fraction. It is not trial-to-trial repeatability.

full_scale_utilisation_axis Maximum absolute scalar axis observation divided by F, a fraction. The denominator is the full peak-to-peak span, not the positive axis limit F/2.

full_scale_utilisation_vm Maximum VM divided by sqrt(3) F, a fraction. It uses the implemented normalising span, not the maximum nonnegative VM supported by a symmetric three-axis sensor.

dead_band, activation_threshold Unestimated NA placeholders.

alignment_lag_s Optional stage cross-correlation lag in seconds; not applied to sample timestamps or errors.

The one-row `overall` table returns `composite_error` (100 times pooled maximum absolute fitted residual divided by sqrt(3) F/2), `response_slope`, `response_intercept` and `scale_factor_error`. Pooled fitting concatenates matched stages, with distinct reference length matching as described above. Diagnostic plots/metadata do not alter fits.

See Also

`clip_dynamic_stages`, `m_dynamic_response`, `measurement-definitions`

`filter_accel_tbl` *Filter axes with a compensated Butterworth filter*

Description

Apply forward-backward Butterworth filtering to each acceleration axis and record processing provenance.

Usage

```
filter_accel_tbl(x, cutoff, forder = 4, ftype = "low")
```

Arguments

`x`	Acceleration table or parser/clip bundle; see `common-data-format`.
`cutoff`	One cutoff for low/high pass, or increasing lower and upper cutoffs for pass/stop bands, in Hz.
`forder`	Positive integer Butterworth order before forward-backward application.
`ftype`	Filter type accepted by `signal::butter`: `"low"`, `"high"`, `"pass"`, `"stop"`.

Details

Adjust cutoffs by $f_c/(\sqrt{2}-1)^{1/(2n)}$, normalise to Nyquist, then apply `signal::filtfilt()`. Adjusted cutoffs must be strictly between zero and Nyquist. The adjustment applies to every supported filter type; effective edges need not match across types. Feature-suite band-pass filtering is separate. Requires finite positive acquisition$fs_derived_hz; no nominal/timestamp fallback. Supply a sufficiently long, contiguous, finite, regularly sampled recording; gaps are not repaired.

Value

The same input container type with filtered axes, unchanged timestamps, appended processing history and invalidated inherited QC. Table and bundle metadata remain synchronised.

Stage bundles

Parser bundles return filtered `data`. Dynamic bundles filter only `full`, leaving `stages`/`schedule` unchanged. Filter first, then reconstruct stages with `clip_dynamic_stages`.

See Also

`clip_dynamic_stages`, `accel_metric_svmbp`, `common-data-format`

invalidate_qc *Remove diagnostics inherited from an earlier signal state*

Description

Remove stored QC after signal changes.

Usage

```
invalidate_qc(x)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.

Value

The same input container with inherited QC removed and attached assessment records marked historical (signal_or_selection_changed); acceleration samples are unchanged.

See Also

attach_qc, common-data-format

measurement-definitions *Calculation definitions and interpretation*

Description

Units, normalisation and implemented arithmetic for the public metrology functions.

Details

Vector magnitude is $v = \sqrt{a_x^2 + a_y^2 + a_z^2}$ in g.

Signed 1-g VM offset bias Initial one-second mean VM minus 1 g, averaged over elapsed time up to and including five minutes. This is a VM offset relative to gravity, not a complete axis calibration.

Signed VM drift rate Ordinary least-squares slope against elapsed hours, g/h. Extrapolation beyond the stationary recording requires an explicitly stated model.

Static noise floor Maximum axis-specific residual SD after linear detrending over the selected stationary window; reporting in mg requires a factor of 1000. The first-five-minute maximum-axis SD is raw, not detrended. Raw sample SD uses denominator n-1; linearly detrended residual SD uses n-2. Static CV is 100 times SD divided by absolute mean, with a near-zero guard.

Allan deviation For non-overlapping cluster means $\bar{x}_j$, $\sigma_A = \sqrt{\text{mean}[(\bar{x}_{j+1} - \bar{x}_j)^2]/2}$. Cluster duration is in seconds and deviation is in g.

Dynamic response and signed VM bias OLS of test VM on length-matched reference VM. Slope is dimensionless; intercept and mean signed test-minus-reference bias are in g. Scale-factor error is slope minus one; multiply it by 100 for percentage reporting. Absolute VM bias is the absolute value of the stage-mean signed difference, not the mean absolute paired difference.

VM integral non-linearity 100 times the maximum absolute fitted residual divided by $\sqrt{3}F$, where F is the test-axis peak-to-peak span in g.

Trial-level composite error 100 times the maximum absolute residual from the pooled response fit divided by $\sqrt{3}F/2$. This denominator differs from stage VM integral non-linearity by a factor of two. The implemented normalising span is not the nonnegative VM output range.

Within-stage epoch-mean VM CV SD of within-stage epoch mean VM divided by their mean. The retained field `repeatability_error_cv` is a fraction. It is distinct from trial-to-trial VM repeatability CV, defined as 100 times SD divided by mean of stage-mean VM across repeated trials for the same unit, stage and pipeline.

Maximum-axis Allan minimum Maximum across axes of their individual minimum Allan deviations over the evaluated averaging times; reporting in micro-g requires a factor of one million. This is distinct from the axis-specific curves.

ENMO Epoch mean of $\max(v-1, 0)$. Negative differences are truncated to zero per sample before averaging.

MAD Epoch mean of $|v - \bar{v}|$, with $\bar{v}$ defined within the epoch. ENMO and MAD are multiplied by 1000 for default mg output.

Index-based dynamic reference matching is not timestamp correction. No equivalence margin or universal fit-for-purpose decision is embedded in these functions. Report the reference, tested range, rate, filtering, epoch settings, signal calibration state, error normalisation and relevant timing diagnostics with every application.

Assessment scope

Establish stationary or synchronous excitation conditions separately. Error estimates are not a complete uncertainty budget. WAM (`https://github.com/miltheo/wam`) supplies cohort aggregation, trial-to-trial repeatability, activation/deadband estimation and inter-device equivalence tests.

References

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: Basic and general concepts and associated terms, JCGM 200:2012. doi:10.59161/JCGM2002012.

IEEE (2018). IEEE Std 2700-2017, Standard for Sensor Performance Parameter Definitions. doi:10.1109/IEEESTD.2018.8277147.

IEEE (2019). IEEE Std 1293-2018, Standard Specification Format Guide and Test Procedure for Linear Single-Axis, Nongyroscopic Accelerometers. doi:10.1109/IEEESTD.2019.8653544.

See Also

- `metronautica-package`.
- `static_metrology_from_tbl`.
- `dynamic_metrology_report`.
- `compute_accel_summaries`.

`m_dynamic_response` *Compute vector-magnitude response and reference-dependent error*

Description

OLS test-on-reference VM response with signed bias and range-normalised residual errors.

Usage

```
m_dynamic_response(test, reference, axis_fsr_g)
```

Arguments

`test`	Test-device vector magnitude in g, or an acceleration table/bundle converted to vector magnitude.
`reference`	Reference vector magnitude in g, or an acceleration table/bundle. Nearest evenly spaced reference indices match the test length.
`axis_fsr_g`	Required finite positive scalar. Configured single-axis peak-to-peak span in g. The VM normalising span is sqrt(3) times this value.

Details

Uses the OLS and nearest-index pairing in `dynamic_metrology_report`. Integral non-linearity = 100 times maximum absolute residual/VM normalising span; composite error uses half that span. The latter represents trial composite error only for complete pooled-trial vectors. No calibration, timestamp correction or uncertainty budget is supplied.

Value

A list with vm_bias_vs_reference (signed mean difference, g), response_slope (dimensionless), response_intercept (g), scale_factor_error (slope minus one), linearity_error_vs_reference and composite_error (percentages using the full and half normalising spans respectively), maximum_absolute_residual_g, and predicted_vm_g (fitted test VM values in g).

See Also

`dynamic_metrology_report`, `measurement-definitions`

Examples

```
reference_vm <- seq(.1, 2, length.out = 100)
test_vm <- .02 + 1.1 * reference_vm
response <- m_dynamic_response(test_vm, reference_vm, axis_fsr_g = 16)
response[c("response_slope", "response_intercept", "composite_error")]
```

`m_load_sample_data` *Retrieve a native recording example into a project*

Description

Copy a local native example or explicitly download it from a configured WAM sample directory, verifying compressed and restored checksums. Measurement values are preserved.

Usage

```
m_load_sample_data(device = "LEAP", type = "static", dest_dir =
    file.path(getwd(), "metronautica-samples"), overwrite = FALSE,
    sample_dir = NULL, include_reference = TRUE,
    base_url = getOption("metronautica.sample_base_url", NULL))
```

Arguments

`device`	One of LEAP, AX3, SENS or GENEActiv; matching is case-insensitive.
`type`	Either static or dynamic.
`dest_dir`	Directory in which the native recording and its provenance sidecar are created when this function is explicitly called.
`overwrite`	Allow replacement of a different existing target file, only after validation. An identical file is reused without accessing its source.
`sample_dir`	Directory containing sample_index.csv and compressed samples. Defaults to the option `metronautica.sample_dir`, if set. Without a local catalogue, the installed package catalogue is used.
`include_reference`	Retrieve the primary HBK reference with dynamic samples. Ignored for static samples.
`base_url`	HTTPS WAM sample directory for samples unavailable locally. Defaults to `metronautica.sample_base_url`. No public URL is preconfigured. A `file:///` directory also supports offline retrieval.

Details

WAM's sample guide defines recording intervals, native-format boundaries, provenance and distribution terms. Clone or download WAM and set `sample_dir` to its included `samples/` folder. The installed catalogue fixes filenames and SHA-256 checksums. Downloads/decompression use temporary files; validation precedes replacement.

Apply each file's own timestamp shift and acquisition settings. Retrieval does not parse, filter or calibrate data. Installation, loading and calculations require neither WAM nor network access. See Getting Started for setup.

Value

The native file path, with a sample_info attribute containing source identity, analysis bounds, units and timestamp shift. A JSON sidecar contains the same information. With a dynamic reference, the path also has reference_file and reference_info attributes.

Example execution

Not run requires external recordings. This retrieval workflow was executed locally for the guide figures.

See Also

Recording examples, parse_raw_auto

Examples

```
## Not run:
options(metronautica.sample_dir = "path/to/wam/samples")
path <- m_load_sample_data("LEAP", "dynamic", dest_dir = tempdir())
attr(path, "sample_info")

## End(Not run)
```

---

parse_raw_auto *Decode a supported accelerometer file*

Description

Detect the device format, decode acceleration in g and retain timestamp, parser and calibration provenance.

Usage

```
parse_raw_auto(path, return = c("data", "bundle"), verbose = FALSE,
    progress = FALSE, time_shift_s = 0, tz_parsed = "UTC",
    hash_sha256 = NULL, qc = TRUE, ...)
```

Arguments

path	Path to one existing device file.
return	Output form: "data" or "bundle".
verbose	Print processing messages.
progress	Show a text progress bar.
time_shift_s	Additive timestamp shift in seconds. No clock-rate correction is inferred.
tz_parsed	Timezone for supported text timestamp decoding and Axivity CWA reading; output is tagged UTC. Binary formats may use device-defined time policies.
hash_sha256	Optional caller-supplied source digest. NULL computes it; a supplied digest is recorded, not independently verified.
qc	Run primary-five diagnostics after decoding (default TRUE). FALSE skips these diagnostics.
...	Named device-specific settings listed under Details. Pass only settings supported by the detected parser.

Details

Supported inputs: GENEActiv/SENS BIN, Axivity CWA/AX3 CSV, ActiGraph GT3X/LEAP CSV/LEAP AGSD and HBK G-Link CSV. Optional decoders are GENEAread, GGIRread (CWA fallback) and read.gt3x; missing dependencies and unsupported formats error without installing packages.

LEAP AGSD uses the shifted-reciprocal conversion verified against paired exports. Supported configuration: LEAP v2, dataVersion 3, firmware 2.1.0.106, 128 Hz, calibrationMethod 2. Other configurations error. Decoding coefficients do not establish independent calibration traceability.

Review timezone assumptions/shifts before pairing. Default QC is stored at meta$processing$qc with ruleset primary-five; standalone sourcing also requires m_qc.R.

Value

An `accel_tbl`, or `list(data = accel_tbl, meta = list)`. Metadata records the source, acquisition settings, decoding/calibration operations and integrity diagnostics. The automatic parser identity is `"m_parsers"` in `meta$file$auto_parser` and `meta$processing$parser_library$name`; `meta$file$parser` identifies the device-specific decoder.

Device-specific arguments through ...

full_scale_range_g Configured symmetric peak-to-peak axis span in g; 16 means -8 to +8 g. Use file metadata when available or supply the documented acquisition setting. It affects range diagnostics and later normalisation, not already decoded CSV acceleration values. Observed minimum/maximum values are not a substitute for sensor limits.

GENEActiv calibrate Logical, default TRUE. Forwarded to GENEAread::read.bin to control upstream coefficient application. This is device-file decoding/calibration, not the within-trial dynamic VM mapping. The returned provenance records whether upstream calibration was requested. mmap_load controls memory-mapped loading, default enabled on 64-bit R, without adding an analysis filter.

SENS scale_g_per_lsb Acceleration scale converting signed axis counts to g; default 0.0078125 from the supported vendor binary specification. A different value requires device-format justification and changes decoded acceleration. No per-file offset or temperature correction is inferred.

GT3X precision Default `"native"` preserves original integer-sample precision through file-scale conversion for supported modern log.bin containers, using the validated read.gt3x 1.2.0 interface. `"legacy_3dp"` selects the earlier upstream scaled/rounded output. Precision and native recording start are recorded in metadata. Native mode does not impute gaps or extract light; choose the explicit legacy mode for those upstream options. Older activity.bin layouts require legacy mode. No lost precision is inferred from CSV or rounded input.

GT3X imputeZeroes Logical, default FALSE: retain the supported upstream decoder's non-imputation route. Enabling it changes the decoded treatment of missing/idle data and must be reported. add_light defaults FALSE; cleanup controls upstream extraction-file cleanup.

Axivity CWA settings use_ggir_fallback defaults TRUE and permits the optional GGIRread path when the fast packed-AX3 layout is unsupported. interpolationType is passed to the fallback reader (default 1). Fast and fallback implementations and their scaling/time policies are retained in metadata; the caller should inspect the selected parser.

Sensor identifiers sensor_vendor and sensor_model optionally annotate the sensor hardware. They do not select conversion coefficients or establish calibration traceability. Only pass arguments accepted by the detected parser.

See Also

common-data-format, qc_accel_tbl, Recording examples

`plot_accel_clip` *Plot triaxial acceleration and optional vector magnitude*

Description

Interactive axis traces with optional VM or non-negative VM-minus-gravity trace.

Usage

```
plot_accel_clip(x, include_vm = FALSE, demean_vm = FALSE, title =
    NULL)
```

Arguments

`x`	Acceleration table or parser/clip bundle; see common-data-format.
`include_vm`	Add sample-level vector magnitude to the triaxial plot.
`demean_vm`	For the optional VM trace, plot max(VM minus 1 g, 0) instead of VM.
`title`	Optional plot title.

Value

An interactive Plotly htmlwidget. Display thinning does not alter the input recording.

See Also

parse_raw_auto, plot_processed_metric

`plot_processed_metric` *Plot an epoch-level acceleration feature*

Description

Interactive display of an existing epoch-level feature.

Usage

```
plot_processed_metric(result, metric, unit = NULL, title = metric)
```

Arguments

`result`	Summary result from compute_accel_summaries or its data_wide table.
`metric`	Name of the numeric column to plot from the processed summary table.
`unit`	Unit label displayed on the processed plot; does not rescale values. NULL uses result$settings$metric_units. Supply a label explicitly for a bare data_wide table or older outputs without this metadata; no mg unit is assumed.
`title`	Optional plot title.

Value

Plotly htmlwidget; values are unchanged.

See Also

compute_accel_summaries, Feature comparison

---

plot_static_allan *Plot axis-specific Allan deviation*

Description

Log-log Allan curves with axis minima and averaging-time labels.

Usage

```
plot_static_allan(allan_df, title_prefix = "Allan deviation")
```

Arguments

allan_df	Output of static_allan: axis, tau (s) and adev (g).
title_prefix	Plot-title prefix.

Details

Both axes use log10 coordinates with inward logarithmic tick marks. Axis minima and their averaging times are labelled from the supplied table; plotting does not recalculate Allan deviation.

Value

ggplot object; data are unchanged.

See Also

static_allan, static_allan_minima_wide

---

plot_static_drift_vm *Plot signed vector-magnitude drift*

Description

One-second VM means and their fitted drift line.

Usage

```
plot_static_drift_vm(d1, drift_fit, title_prefix = "Static drift")
```

Arguments

d1	One-second means from static_1hz_means, including elapsed_h (hours since the first populated UTC-second bin).
drift_fit	The fitted linear model, obtained as static_drift(d1)$fit_vm.
title_prefix	Plot-title prefix.

Value

ggplot object; data are unchanged.

See Also

static_1hz_means, static_drift

---

qc_accel_tbl *Quantify signal and timestamp integrity*

Description

Report integrity diagnostics without sorting, repairing or removing samples.

Usage

```
qc_accel_tbl(x)
```

Arguments

x	Acceleration table or parser/clip bundle. Requires exactly time (POSIXct), ax_g, ay_g, az_g (numeric g), in that order; metadata precedence follows common-data-format.

Details

The primary-five rules follow Supplementary Table S2 of the research article under submission.

1. **Data integrity** Count non-finite timestamps. Count repeated finite timestamps after their first occurrence. Count negative differences between adjacent finite timestamps. Count non-finite acceleration values across X, Y and Z.

   Reported output: Four separate counts; total rows and total axis values retained as denominators.

   Source: Study-defined availability and ordering checks.

2. **Sample timing** Select finite, positive differences between adjacent timestamps. Report their median, interquartile range (IQR) and maximum.

   Reported output: Typical spacing, spacing spread and longest spacing (ms).

   Source: Study-defined application; IQR is an established measure of spread (NIST/SEMATECH, n.d.).

3. **Axis-zero proportion** For each axis, divide the number of exact-zero values by the number of finite values and multiply by 100.

   Reported output: Zero values (%) separately for X, Y and Z.

   Source: Study-defined descriptive diagnostic.

4. **Unusual cross-axis patterns** Count samples with $X = Y = Z = 0$. Count samples with $X = Y = Z$ and $X \neq 0$. Flag each pattern separately when it occurs in at least two consecutive eligible samples.

   Reported output: Sample count and sequence-present flag for each pattern.

   Source: Adapted from NHANES all-zero and equal-axis flags (CDC/NCHS, 2022); the two-sample all-zero minimum is study-defined.

5. **Near-limit values** Pool the finite acceleration values from X, Y and Z. Report the percentage whose absolute value is at least $0.98L$, where L is the configured positive axis limit.

   Reported output: Near-limit values (%); retain L, threshold, numerator and denominator.

   Source: NHANES range-limit concept (CDC/NCHS, 2022); 98% and pooling are study-defined. Span terminology follows JCGM (2012, entries 4.4–4.5).

Finite excludes missing values, NaN and infinities. Unavailable quantities are reported as NA. IQR requires at least two positive intervals and uses linearly interpolated quartiles. For symmetric limits, L is half the configured full-scale span. Eligible pattern samples have finite timestamps and XYZ values. Consecutive samples are adjacent recorded rows with strictly increasing timestamps; an ineligible row or non-increasing step breaks a sequence. The equal non-zero value may vary between samples. No gap-duration cutoff is imposed. The intended all-axis NHANES zero condition is used, not its documented X-only implementation error; the original minute-level flag algorithm is not reproduced.

Value

A named list of scalar diagnostics. Counts are sample counts except `invalid_axis`, `near_limit_n` and `finite_axis_n`, which count axis observations. Percentages are on the 0–100 scale.

References

NIST/SEMATECH (n.d.). e-Handbook of Statistical Methods, Section 1.3.5.6: Measures of Scale. https://www.itl.nist.gov/div898/handbook/eda/section3/eda356.htm.

CDC/NCHS (2022). Data Quality Flag Summary Table for the Physical Activity Monitor (PAM) Data Collected in NHANES 2011–2014 and NNYFS. https://wwwn.cdc.gov/nchs/nhanes/Pam/.

JCGM (2012). International vocabulary of metrology, JCGM 200:2012, entries 4.4–4.5. doi:10.59161/JCGM2002012.

See Also

attach_qc, common-data-format

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
q <- qc_accel_tbl(x)
q[c("n_samples", "median_dt_ms", "near_limit_pct")]
```

static_1hz_means *Compute one-second triaxial and vector-magnitude means*

Description

Axis and sample-VM means in populated UTC-second bins.

Usage

```
static_1hz_means(x)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.

Value

A tibble with sec (UTC second-bin start), ax_mean, ay_mean, az_mean and vm_mean (one-second means in g), and elapsed_h (hours since the first populated bin).

Method

Floor Unix timestamps to seconds. Average each axis and sample VM within populated bins, omitting missing values; leave empty seconds unfilled. elapsed_h starts at the first populated bin.

See Also

static_drift, static_offset_vm_5min

static_allan *Compute axis-specific non-overlapping Allan deviation*

Description

Axis-wise Allan deviation over supported averaging times.

Usage

```
static_allan(x)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.

Value

Tibble: axis, tau (s), adev (g).

Method

For each axis, request 60 log-spaced averaging times from 0.1 to 10000 s through allan_nonoverlap; at least four clusters are required. Rate is estimated from positive timestamp intervals. Requires uniformly sampled stationary data; gaps are neither repaired nor resampled.

See Also

allan_nonoverlap, static_allan_minima_wide, plot_static_allan

---

static_allan_minima_wide

Summarise axis-specific Allan minima

Description

Minimum evaluated Allan deviation and corresponding averaging time for each axis.

Usage

```
static_allan_minima_wide(allan_df)
```

Arguments

allan_df	Output of static_allan: axis, tau (s) and adev (g).

Value

A one-row tibble with x_allan_min_g, y_allan_min_g and z_allan_min_g (axis minima in g), plus x_cluster_time_s, y_cluster_time_s and z_cluster_time_s (the corresponding evaluated averaging times in seconds).

Method

Find the smallest evaluated deviation separately for X, Y and Z and retain its averaging time. A maximum-axis Allan minimum is obtained separately by taking the maximum of these three axis minima.

See Also

static_allan, measurement-definitions

`static_axis_noise_metrics` *Compute axis-specific stationary noise measures*

Description

Raw, detrended and initial-window axis/VM variability.

Usage

```
static_axis_noise_metrics(x, cv_min_abs_mean_g = 0.005)
```

Arguments

`x`	Acceleration table or parser/clip bundle; see `common-data-format`.
`cv_min_abs_mean_g`	Absolute-mean threshold (g) below which CV is NA; default 0.005 g guards near-zero division.

Value

One-row tibble with raw and linearly detrended sample SDs, first-five-minute raw SDs, CVs, ranges and maximum-/median-axis summaries. Acceleration fields use g unless their name explicitly specifies mg; CV fields are percentages.

Returned fields

For each prefix x, y, z and vm, the one-row output contains `n` (finite samples used), `mean_g`, `sd_g` (sample SD), `detrended_sd_g` (linear-residual SD), `sd_5min_g` (raw SD within the first five minutes, inclusive boundary), `min_g`, `max_g`, `range_g`, `cv_percent` and `detrended_cv_percent`. Each prefix is joined to the field name by an underscore. Linear-residual SD uses n-2; raw SD uses n-1.

`max_axis_sd_g` and `median_axis_sd_g` summarise raw X/Y/Z SDs; `max_axis_noise_sd_g` and `median_axis_noise_sd_g` summarise detrended X/Y/Z SDs. Their maximum-axis identifiers end in `_axis`; analogous `sd_5min` fields use the initial raw window. `static_n_samples` counts supplied rows.

`static_noise_floor_mg` is 1000 times maximum-axis detrended SD. The retained `static_noise_floor_5min_mg` field is instead 1000 times initial raw maximum-axis SD; it is not detrended. `static_raw_max_axis_sd_mg` is the full-window raw maximum-axis SD in mg. `vm_sd_mg`, `vm_noise_sd_mg` and `vm_sd_5min_mg` give VM raw, detrended and initial raw SDs in mg; `vm_noise_cv_percent` is detrended VM CV. These operational outcomes do not independently establish stationarity.

See Also

`static_noise_summary_from_tbl`, `measurement-definitions`

static_clip *Select a stationary assessment interval*

Description

Select an explicit time window and recompute QC.

Usage

```
static_clip(x, start_utc, duration_h)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.
start_utc	UTC window start; POSIXct or parseable text.
duration_h	Required positive duration (h), with exclusive end. processing$static_window records requested bounds, observed span and boundary coverage. Incomplete boundary coverage warns; internal gaps and stationarity are not tested.

Value

The selected acceleration table; the end of the static interval is excluded.

Container behaviour

Returns an acceleration table for both table and bundle input. A supplied parser bundle is unwrapped from data; a dynamic bundle is unwrapped from full. Metadata is attached to the selected table; no stage bundle is returned or updated. Each selected interval receives freshly computed QC.

See Also

static_metrology_from_tbl, clip_bundle

static_drift *Estimate signed drift from one-second means*

Description

Signed OLS slopes of one-second axis and VM means against elapsed hours.

Usage

```
static_drift(d1)
```

Arguments

d1	One-second means from static_1hz_means, including elapsed_h (hours since the first populated UTC-second bin).

Value

A list with `vm_drift_g_per_h`, `x_drift_g_per_h`, `y_drift_g_per_h` and `z_drift_g_per_h`: signed slopes in g/h. `fit_vm` is the fitted VM `lm` object. Fitted axis-model objects are not returned.

Method

Fit separate OLS models of one-second axis/VM means against elapsed_h. Requires a stationary interval; a slope fitted to uncontrolled movement does not establish sensor drift.

See Also

`static_1hz_means`, `plot_static_drift_vm`, `measurement-definitions`

`static_metrology_from_tbl`
Characterise a stationary recording

Description

Estimate signed 1-g VM offset bias, drift rate, static noise floor and non-overlapping Allan deviation.

Usage

```
static_metrology_from_tbl(x, instrument = NA_character_, device_id =
    NA_character_, device_vendor = NA_character_, device_model =
    NA_character_, fs_nominal_hz = NA_real_, fs_derived_hz =
    NA_real_, out_dir = NULL, start_utc, duration_h,
    title_prefix = NULL, verbose = FALSE, progress = TRUE,
    attach_report = TRUE)
```

Arguments

`x`	Acceleration table or parser/clip bundle; see `common-data-format`.
`instrument`	Instrument label for summaries/filenames; defaults to available vendor/model metadata.
`device_id`	Summary/filename unit label from acquisition$device_id; does not select calibration coefficients.
`device_vendor`	Manufacturer label from acquisition metadata; affects neither decoding nor calculations.
`device_model`	Model label from acquisition metadata; affects neither decoding nor calculations.
`fs_nominal_hz`	Nominal frequency label (Hz); defaults to acquisition metadata. Calculations retain their timestamp-derived rate.
`fs_derived_hz`	Derived frequency label (Hz); defaults to metadata/timestamps, without overriding calculation windows.
`out_dir`	Output directory for CSV, RDS and diagnostic plots; NULL keeps results in memory.

start_utc	UTC window start; POSIXct or parseable text.
duration_h	Required positive duration (h), with exclusive end. processing$static_window records requested bounds, observed span and boundary coverage. Incomplete boundary coverage warns; internal gaps and stationarity are not tested.
title_prefix	Plot-title prefix.
verbose	Print processing messages.
progress	Show a text progress bar.
attach_report	TRUE (default) appends a compact assessment to processing$reports$static on returned data and clipped_data. FALSE adds no record. Capture the returned data; the caller's object is not modified in place.

Details

Stationarity must be established separately. The initial five-minute SD is raw; the full-window noise floor is detrended. measurement-definitions defines the outcomes and normalisation.

Value

An invisible list: summary (one-row table), allan (cluster durations and deviations), drift_plot, allan_plot, clipped_data, one_hz (axis/VM means and elapsed hours), meta, files (written paths, if requested), and data (the complete input table with optional attached assessment). meta describes clipped_data; accel_meta(data) describes the complete input. Attached results include summary, Allan and one-Hz tables; shared record fields and retention follow common-data-format.

Summary fields

The one-row summary includes instrument/unit identifiers, actual first-to-last sample span in trial_duration_h, four *_drift_g_per_h slopes, vm_offset_g_5min, nominal/derived rate labels in Hz, and observed clip_start_utc/clip_end_utc. Axis/VM noise columns follow static_axis_noise_metrics. Axis-specific Allan minima and their evaluated averaging times follow static_allan_minima_wide. Actual sample span differs from requested window duration by endpoint sampling and any incomplete coverage; consult meta$processing$static_window.

See Also

Recording examples

---

static_noise_summary_from_tbl

Summarise stationary axis noise

Description

Noise and variability for an explicitly selected stationary interval.

Usage

```
static_noise_summary_from_tbl(x, instrument = NA_character_,
    device_id = NA_character_, device_vendor = NA_character_,
    device_model = NA_character_, fs_nominal_hz = NA_real_,
    fs_derived_hz = NA_real_, out_dir = NULL, start_utc, duration_h,
    verbose = FALSE, progress = TRUE, attach_report = TRUE)
```

Arguments

x	Acceleration table or parser/clip bundle; see common-data-format.
instrument	Instrument label for summaries/filenames; defaults to available vendor/model metadata.
device_id	Summary/filename unit label from acquisition$device_id; does not select calibration coefficients.
device_vendor	Manufacturer label from acquisition metadata; affects neither decoding nor calculations.
device_model	Model label from acquisition metadata; affects neither decoding nor calculations.
fs_nominal_hz	Nominal frequency label (Hz); defaults to acquisition metadata. Calculations retain their timestamp-derived rate.
fs_derived_hz	Derived frequency label (Hz); defaults to metadata/timestamps, without overriding calculation windows.
out_dir	Output directory for CSV, RDS and diagnostic plots; NULL keeps results in memory.
start_utc	UTC window start; POSIXct or parseable text.
duration_h	Required positive duration (h), with exclusive end. processing$static_window records requested bounds, observed span and boundary coverage. Incomplete boundary coverage warns; internal gaps and stationarity are not tested.
verbose	Print processing messages.
progress	Show a text progress bar.
attach_report	TRUE (default) appends the noise assessment to processing$reports$static on returned data and clipped_data. FALSE adds no record. The caller's object is not modified in place.

Value

Invisible list: summary (one-row noise/identifier table), clipped_data, meta (including window coverage), files (written paths or NULL), and data (the complete input table with optional attached assessment). Attached results contain the noise summary; shared record fields and retention follow common-data-format.

Summary fields

Noise and variability fields follow static_axis_noise_metrics; the report adds instrument/unit labels, nominal/derived frequency labels in Hz, actual first-to-last sample duration in hours and observed clip bounds. The selected data and boundary-coverage metadata are returned separately. This report does not calculate drift, initial VM offset or Allan curves; use static_metrology_from_tbl for those outcomes.

See Also

static_axis_noise_metrics, static_metrology_from_tbl

`static_offset_vm_5min` *Estimate signed one-g vector-magnitude offset bias*

Description

Initial five-minute mean VM minus 1 g.

Usage

```
static_offset_vm_5min(d1)
```

Arguments

d1	One-second means from static_1hz_means, including elapsed_h (hours since the first populated UTC-second bin).

Value

Signed initial-window VM offset in g relative to 1 g.

Method

Select one-second bins satisfying elapsed_h <= 5/60, average vm_mean and subtract 1 g. The inclusive elapsed-bin rule is retained exactly. This measures VM offset relative to gravity, not an axis-specific multi-orientation calibration.

See Also

static_1hz_means, measurement-definitions

Index

accel_meta, 4
accel_metric_agcounts, 4, *26*
accel_metric_ai, 6, *26, 27*, *29*
accel_metric_enmo, 8, *27*
accel_metric_mad, 9, *27*
accel_metric_mims, 10, *25–27*
accel_metric_odba, 11, *25*, *27*
accel_metric_rocam, 13, *25*, *27*
accel_metric_svmbp, 14, *25*, *27*, *35*
accel_metric_svmg, 15, *25*, *27*
accel_metric_vm, 16, *27*
accel_metric_z_angle, 16, *26, 27*
accel_metric_z_angle_change, 17, *26, 27*
allan_nonoverlap, 18, *47*
as_accel_tbl, *4*, 19, *23*
attach_qc, 20, *21*, *29, 30*, *36*, *45*

clip_bundle, 20, *22*, *49*
clip_dynamic_stages, *21*, 21, *34, 35*
common-data-format, 22
compute_accel_summaries, *6–9*, *11*, *13–18*, *23*, 23, *24*, *37*, *43*
compute_ai_noise_var_from_static, *7*, 28

dm_apply_vm_calibration_bundle, *21*, 29, *31*
dm_apply_vm_calibration_tbl, 30, *31*
dm_fit_vm_calibration, *29, 30*, 30
dynamic_metrology_report, *22*, 31, *37, 38*

feature-comparison *(*metronautica-package*)*, 3
filter_accel_tbl, *22*, 35

getting-started *(*metronautica-package*)*, 3

implementation-guide *(*metronautica-package*)*, 3
invalidate_qc, *20*, 36

m_dynamic_response, *34*, 38
m_load_sample_data, 39
measurement-definitions, 36
metronautica *(*metronautica-package*)*, 3
metronautica-package, 3
metronautica-tibble *(*common-data-format*)*, 22

native-recording-examples *(*metronautica-package*)*, 3
native-study-examples *(*metronautica-package*)*, 3

parse_raw_auto, *6*, *40*, 40, *42*
plot_accel_clip, 42
plot_processed_metric, *42*, 42
plot_static_allan, 43, *47*
plot_static_drift_vm, 43, *50*

qc_accel_tbl, *6*, *20*, *23*, *42*, 44

static_1hz_means, *44*, 46, *49, 50*, *53*
static_allan, *18*, *43*, 46, *47*
static_allan_minima_wide, *43*, *47*, 47, *51*
static_axis_noise_metrics, *29*, 48, *51–53*
static_clip, *21*, 49
static_drift, *44*, *46*, 49
static_metrology_from_tbl, *37*, *49*, 50, *52, 53*
static_noise_summary_from_tbl, *48*, 51
static_offset_vm_5min, *46*, 53